

Orchestrating Healthcare ETL: An Apache Airflow Pipeline for FHIR Data Transformation

Author: **Vedant Hirekar**

Course: **Management, Access and Use of Big Data**

1. Introduction

Healthcare data interoperability remains a critical challenge in modern healthcare systems. The HL7 FHIR (Fast Healthcare Interoperability Resources) standard, while providing a robust framework for healthcare data exchange, presents significant barriers for data analysts and business intelligence teams due to its complex nested JSON structure. This project addresses the fundamental question: how can we efficiently transform complex FHIR bundles into analytics-ready tabular formats at scale?

Focusing on the Data Pipelines and Workflow Orchestration module, this project implements an end-to-end automated ETL pipeline that detects incoming FHIR patient bundles, extracts information, and outputs structured CSV files ready for downstream analytics and visualization tools like Tableau or Power BI.

2. Background

Why This Matters for Big Data

Healthcare data exemplifies the four V's of big data. A single patient's FHIR bundle can contain 1-50MB of deeply nested JSON with thousands of resources including observations, medications, procedures, and encounters. Healthcare systems processing millions of patients generate terabytes of data annually, arriving continuously from EHRs, labs, and imaging systems. The semi-structured nature of FHIR data requires sophisticated parsing techniques, while common data quality issues demand robust validation and error handling.

Course Concepts Applied

This project explicitly applies three core concepts from the course:

1. Pipeline Orchestration (Apache Airflow): Airflow provides DAG-based workflow management, enabling complex task dependencies, retry logic, and monitoring. The pipeline implements a multi-stage ETL workflow with proper error handling and complete data lineage tracking.

2. Data Pipeline Design Patterns: The implementation follows industry best practices including staging/processing folder patterns to prevent race conditions, data archiving for audit trails, and idempotent operations for reliability, ensuring the pipeline is production-ready and maintainable.

3. Distributed Data Processing (PySpark): The pipeline uses Apache Spark's distributed computing framework to process large FHIR bundles in parallel. Spark's DataFrame API enables efficient schema inference, nested field extraction, and columnar operations on JSON data, addressing scalability challenges when processing thousands of patient records.

3. Methodology

3.1 Environment and Platform

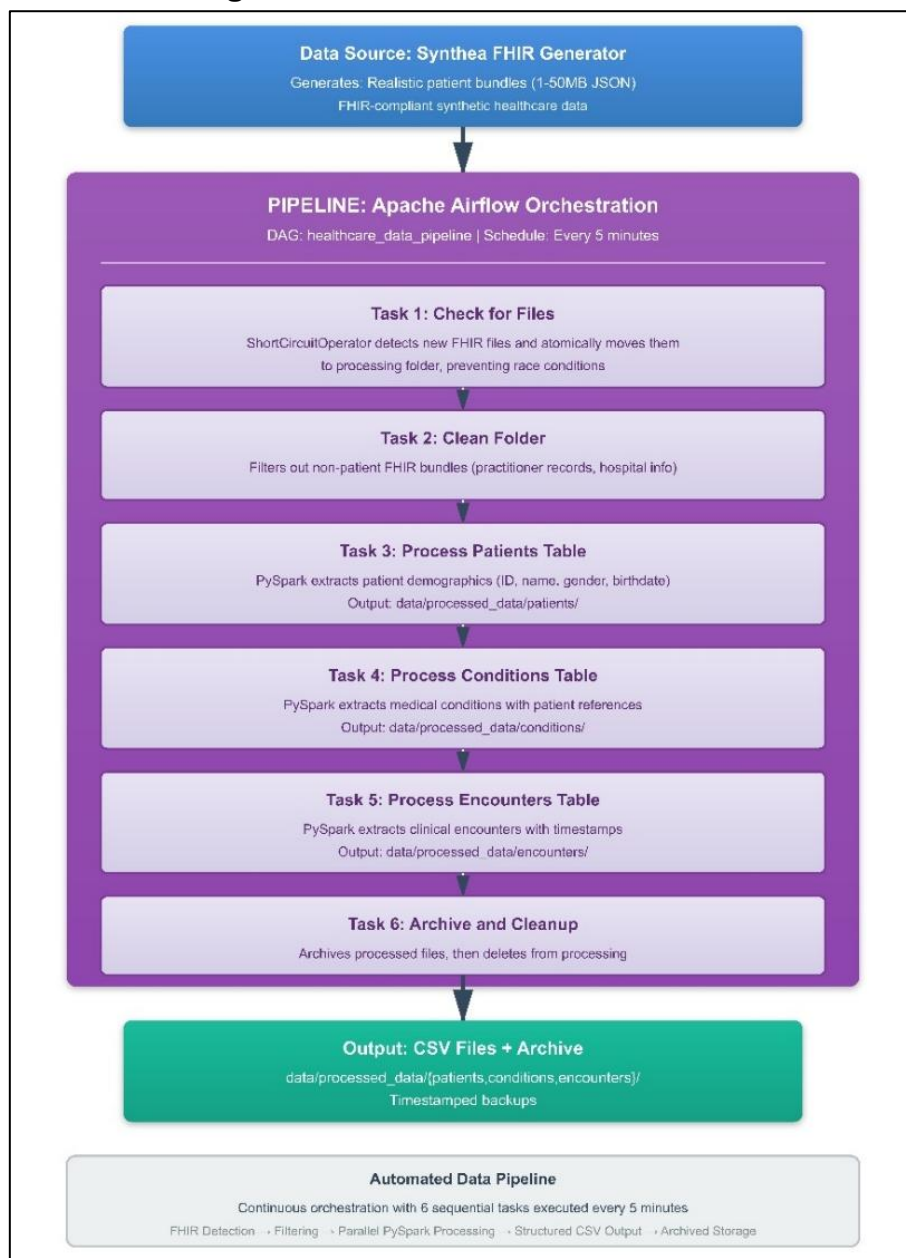
Platform: Local development environment using Docker container on Windows 11

Infrastructure: Docker application, using the official Apache Airflow image alongside a custom Dockerfile configured with Java and Spark for executing ETL scripts.

Key Technologies:

- Apache Airflow 3.1.2 (CeleryExecutor)
- Apache Spark 3.5.0 (PySpark)
- Python 3.13
- Docker Compose for container orchestration
- Java 17 (Spark dependency)

3.2 Architecture and Design



3.3 Implementation Steps

Step 1 Custom Docker Environment: Created a Dockerfile extending Apache Airflow 3.1.2 with Java 17 and PySpark 3.5.0. Configured volume mounts for dags/, scripts/, and data/ folders.

```
PS D:\Ved\airflow-docker> docker-compose up -d
[+] Running 9/9
✔ Network airflow-docker_default          Created
0.1s
✔ Container airflow-docker-postgres-1    Healthy
69.0s
✔ Container airflow-docker-redis-1       Healthy
69.0s
✔ Container airflow-docker-airflow-init-1 Exited
68.8s
✔ Container airflow-docker-airflow-triggerer-1 Starting
67.7s
✔ Container airflow-docker-airflow-apiserver-1 Healthy
134.6s
✔ Container airflow-docker-airflow-scheduler-1 Starting
67.8s
✔ Container airflow-docker-airflow-dag-processor-1 Starting
67.8s
✔ Container airflow-docker-airflow-worker-1 Started
134.7s
```

Step 2 ETL pipeline: It comprises six stages: detecting and isolating new FHIR files, cleaning non-patient data, and running PySpark jobs to extract patient demographics, conditions, and encounters. Finally, processed files are archived and removed from the processing directory.

Step 3 Race Condition Prevention: Implemented staging/processing folder pattern where incoming data lands in data/generated_data/fhir/, pipeline atomically moves files to data/generated_data/processing/, and all processing tasks read from the isolated processing folder. New data arriving during processing doesn't interfere with the current batch.

Step 4 Processing: Each script initializes a Spark session, reads the FHIR JSON data, flattens the nested structure to extract resource elements, filters for Patient Resource, selects key attributes like ID, gender, and birth date, and appends the transformed data as CSV files.

3.4 Data Description

Data Source: Synthea - Synthetic Patient Generator producing realistic HL7 FHIR R4 data

Characteristics:

- Format: HL7 FHIR R4 JSON bundles
- Size: 1-50 MB per patient bundle
- Structure: Nested JSON with 10-50 resource entries per bundle

Example of a few lines from one patient file –

```
data > generated_data > fhir > 11_Amy965_Uda218_Regahe09_08c20f9-98b9-462b-f3af-0a1a5b06e42.json > {} entry > {} > {} resource > {} extension
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
{
  "resourceType": "Bundle",
  "type": "transaction",
  "entry": [ {
    "fullUrl": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-patient",
    "resource": {
      "resourceType": "Patient",
      "id": "86c5f9-98b9-462b-f3af-0a1a5b06e42",
      "meta": {
        "profile": [ "http://hl7.org/fhir/us/core/StructureDefinition/us-core-patient" ]
      },
      "text": {
        "status": "generated",
        "div": "div xmlns='http://www.w3.org/1999/xhtml'>generated by ca href='https://github.com/synthetichealth/synthea'>Synthe
      },
      "extension": [ {
        "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-race",
        "extension": [ {
          "url": "code",
          "valueCoding": {
            "system": "urn:oid:2.16.840.1.113883.6.238",
            "code": "2028-9",
            "display": "Asian"
          }
        }, {
          "url": "text",
          "valueString": "Asian"
        }
      ],
        "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-ethnicity",
        "extension": [ {
          "url": "code",
          "valueCoding": {
            "system": "urn:oid:2.16.840.1.113883.6.238",
            "code": "2186-5",
            "display": "Not Hispanic or Latino"
          }
        }
      ]
    }
  } ]
}
```

3.5 Validation Plan

Validation Metrics:

1. Pipeline Reliability: Run pipeline multiple times on same data to verify idempotent operations
2. Performance: Track task duration in Airflow UI metrics and logs
3. Race Condition Prevention: Generate new data during pipeline execution to verify isolation
4. Schema Correctness: Validate output CSV structure with Spark schema validation

4. Results

4.1 Pipeline Execution Success

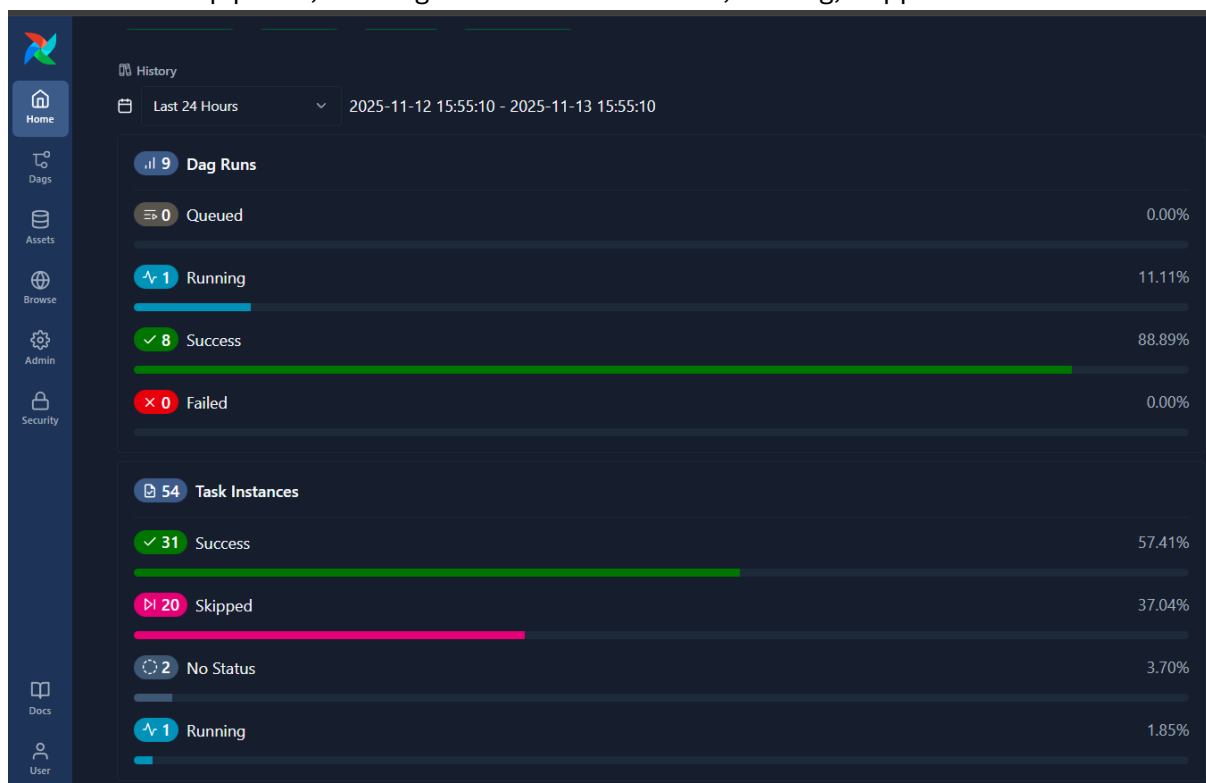
What: All six pipeline tasks executed successfully across multiple DAG runs

How: Monitored via Airflow UI task view and execution logs

So What: The pipeline demonstrates reliability with proper error handling. The ShortCircuitOperator correctly skips downstream tasks when no files are present, preventing unnecessary resource consumption and failed task states.

Below are screenshots demonstrating the results -

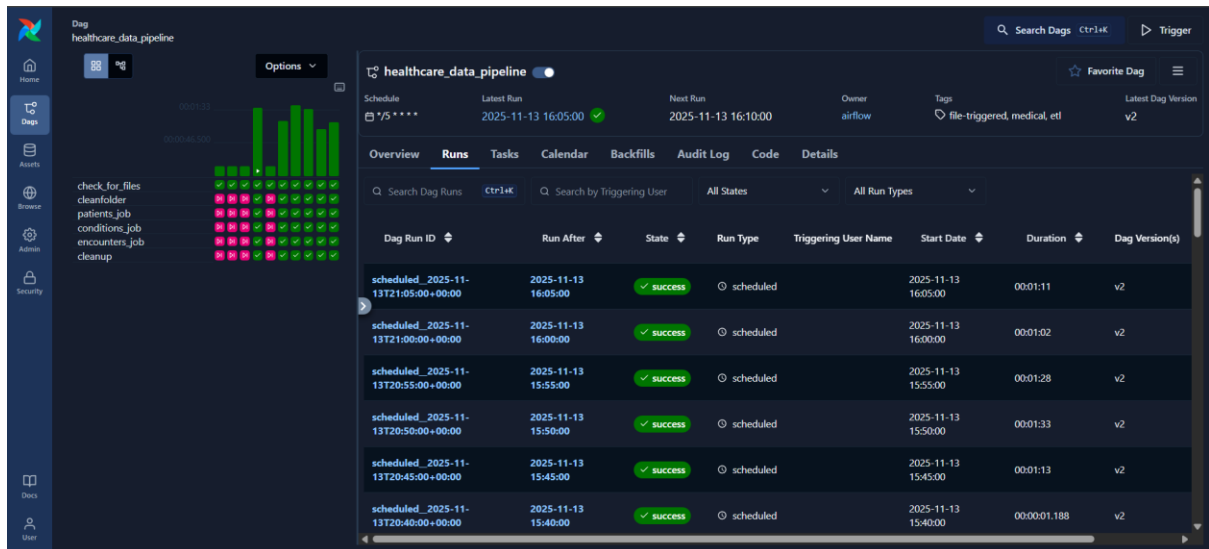
Overview of the pipeline, showing no. of tasks successful, running, skipped and failed.



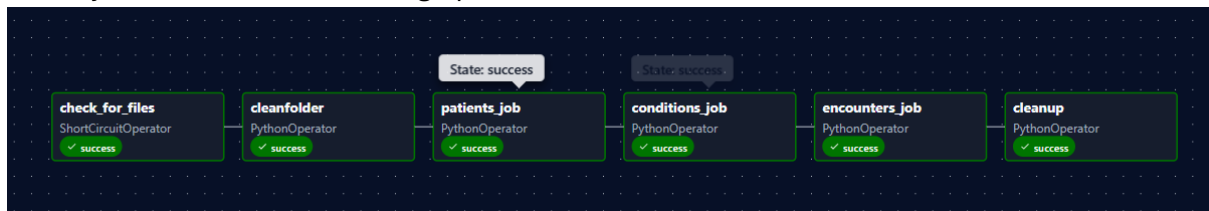
Below we can see details of the DAG **healthcare_data_pipeline**.

On the left, we can see the 6 tasks and their status in every run. Green means success and pink means the tasks were skipped because there was no new data in the folder.

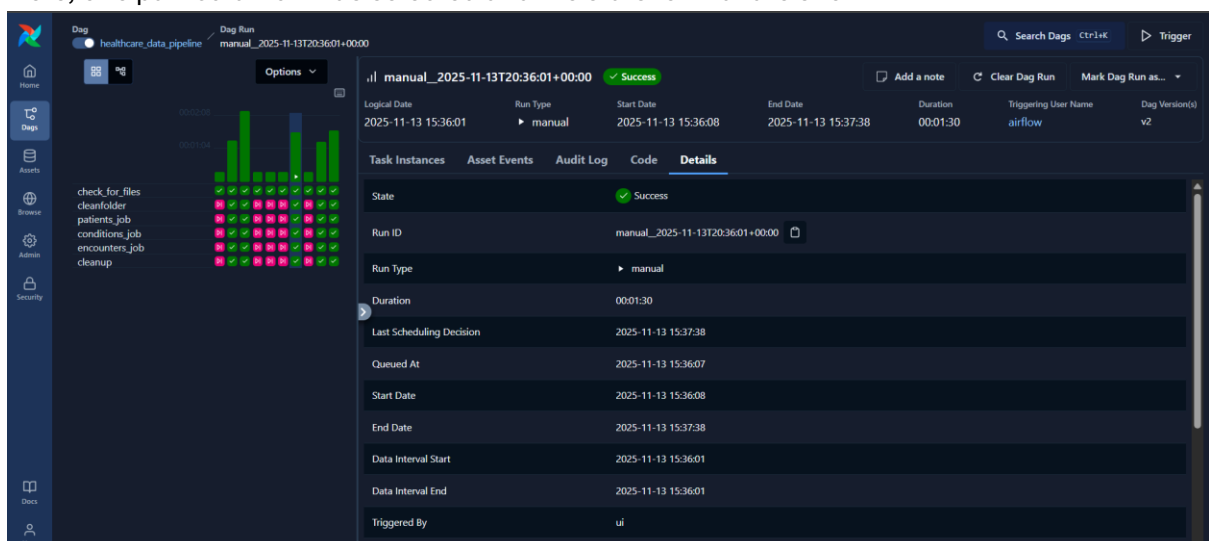
On the right, we can see if the overall run for that particular instances was successful or not. We can see that after every 5 minutes the DAG triggers and it takes around 1-1.5 minutes to complete the whole flow.



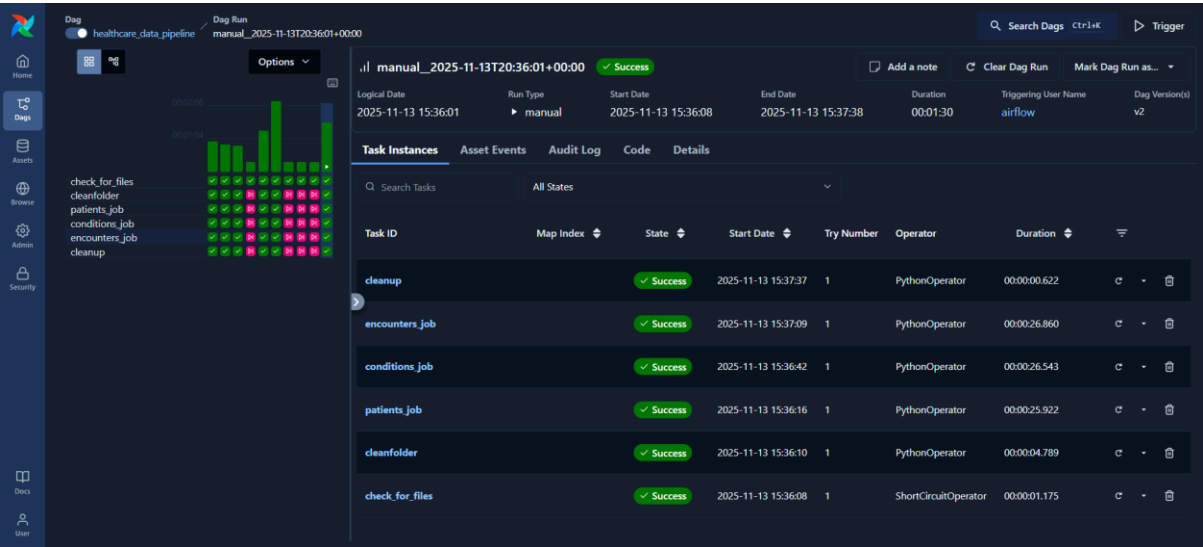
Below you can see the tasks in a graph format.



Here, one particular run was selected and the stats for that are shown



Here we can see, how much time each task takes. The scripts for conversion take around 25 seconds each.



4.2 Data Transformation Results

- What:** Successfully extracted and transformed 50+ patient bundles into structured CSV files
- How:** PySpark processing with automatic schema inference and nested field extraction using SQL expressions
- So What:** The transformation achieves the extraction of crucial information. The resulting CSV format is immediately usable in analytics tools like Tableau, Power BI, or Python pandas without additional preprocessing.

Below are metrics and screenshots of conversion.

Table 1: Data Transformation Metrics for a few cycles (16 patient files)

Metric	Raw FHIR	Processed CSV	Improvement
Total Size	142 MB	500 KB	284x smaller
File Count	16 bundles	33 CSV partitions	Distributed output
Avg. File Size	8.9 MB	15 KB	Highly compressed
Schema Complexity	Nested JSON (5+ levels)	Flat table (4-5 columns)	Analytics-ready

You can refer to how raw data looks like in section 3.4

After processing the data is stored in parts in CSV format. Spark stores data in parts to be able to parallelise the tasks. These parts can easily be read by Visualization tools like PowerBI and Tableau.

Below you can see one part of the patients CSV file.

	A	B	C	D
1	id	gender	birthDate	full_name
2	a93c39d7-c5fb-30e2-d92a-439776544f0f	female	26-09-1952	Elizbeth263 Veum823
3				

One Part of conditions CSV file

A		B	C	D
1	id	patient_ref	condition_name	recorded_date
2	4051b4fb-8e21-3f7e-2e9d-c0dfcd208b4f	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Risk activity involvement (finding)	1974-09-23T17:15:12-05:00
3	4051b4fb-8e21-3f7e-616c-ac691efb7e7d	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Educated to high school level (finding)	1976-10-04T17:31:19-05:00
4	4051b4fb-8e21-3f7e-a4e4-d7f7bfe7e639	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	History of tubal ligation (situation)	1983-10-24T18:45:55-05:00
5	4051b4fb-8e21-3f7e-859b-7ba14b4e9174	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Prediabetes (finding)	1986-10-20T16:32:55-05:00
6	4051b4fb-8e21-3f7e-41fd-e18e01f5dab4	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Anemia (disorder)	1989-10-23T16:32:55-05:00
7	4051b4fb-8e21-3f7e-ce48-d221ec058e1f	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Has a criminal record (finding)	1989-10-23T17:26:00-05:00
8	4051b4fb-8e21-3f7e-a425-cb5d3071751e	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Essential hypertension (disorder)	2004-08-30T16:32:55-05:00
9	4051b4fb-8e21-3f7e-fbdb-98cf17bd9a60	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Chronic kidney disease stage 1 (disorder)	2011-07-11T17:32:55-04:00
10	4051b4fb-8e21-3f7e-a267-3478ee80a579	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Disorder of kidney due to diabetes mellitus (disorder)	2011-07-11T17:32:55-04:00
11	4051b4fb-8e21-3f7e-50fa-a3168452d11d	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Chronic pain (finding)	2014-02-05T16:32:55-05:00
12	4051b4fb-8e21-3f7e-cedd-000cf29c0f28	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Chronic low back pain (finding)	2014-02-05T16:32:55-05:00
13	4051b4fb-8e21-3f7e-5cd6-523d2c3ceb86	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Chronic neck pain (finding)	2014-02-05T16:32:55-05:00
14	4051b4fb-8e21-3f7e-3ef1-ae84d259a11a	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Chronic kidney disease stage 2 (disorder)	2014-03-24T17:32:55-04:00
15	4051b4fb-8e21-3f7e-0882-bead3b89d182	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Microalbuminuria due to type 2 diabetes mellitus (disorder)	2014-03-24T17:32:55-04:00
16	4051b4fb-8e21-3f7e-81da-038b055d1ea0	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Opioid abuse (disorder)	2014-03-26T00:32:55-04:00
17	4051b4fb-8e21-3f7e-63d6-a9d478371bd2	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Victim of intimate partner abuse (finding)	2015-03-23T19:00:07-04:00
18	4051b4fb-8e21-3f7e-b44a-16a9495b86c4	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Unemployed (finding)	2015-06-22T18:09:08-04:00
19	4051b4fb-8e21-3f7e-b80c-a93f3419ae72	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Medication review due (situation)	2015-11-16T16:32:55-05:00
20	4051b4fb-8e21-3f7e-beda-63f2529afc66	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Medication review due (situation)	2015-11-30T16:32:55-05:00
21	4051b4fb-8e21-3f7e-c27b-bf3142bb177b	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Stress (finding)	2015-11-30T17:25:41-05:00
22	4051b4fb-8e21-3f7e-9487-f3b3d57ea6d8	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Medication review due (situation)	2015-12-14T16:32:55-05:00
23	4051b4fb-8e21-3f7e-add8-9ba384558412	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Gingivitis (disorder)	2015-12-14T17:46:12-05:00
24	4051b4fb-8e21-3f7e-72c2-399e897c10aa	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Medication review due (situation)	2015-12-21T16:32:55-05:00
25	4051b4fb-8e21-3f7e-7aa6-23663ed19b76	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Full-time employment (finding)	2015-12-28T17:13:10-05:00
26	4051b4fb-8e21-3f7e-5905-301169e99975	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Stress (finding)	2016-01-11T17:28:37-05:00
27	4051b4fb-8e21-3f7e-c9ce-ea729f15270	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Social isolation (finding)	2016-01-25T17:26:24-05:00
28	4051b4fb-8e21-3f7e-859b-7ba14b4e9174	urn:uuid:4051b4fb-8e21-3f7e-6c69-c9d7f23be5d0	Prediabetes (finding)	1986-10-20T16:32:55-05:00

One Part of encounters CSV file

A		B	C	D	E
1	id	patient_ref	class_code	start_date	end_date
2	ccb532ac-1308-fa17-a34c-f121e0848988	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1959-08-30T03:38:41-05:00	1959-08-30T03:53:45-05:00
3	ccb532ac-1308-fa17-bdb5-09f19dc190d4	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1960-09-04T03:38:41-05:00	1960-09-04T04:31:19-05:00
4	ccb532ac-1308-fa17-25b8-db1548f9f03e	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1967-09-17T03:38:41-05:00	1967-09-17T04:30:17-05:00
5	ccb532ac-1308-fa17-f049-fa718cc5e8f9	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1973-09-23T03:38:41-05:00	1973-09-23T04:30:18-05:00
6	ccb532ac-1308-fa17-e695-02b9e74b1807	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1976-09-26T03:38:41-05:00	1976-09-26T04:19:46-05:00
7	ccb532ac-1308-fa17-fb67-da68f22a21d7	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1976-10-03T03:38:41-05:00	1976-10-03T04:58:18-05:00
8	ccb532ac-1308-fa17-f2e5-e4942df8dac8	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1981-01-25T03:38:41-05:00	1981-01-25T04:29:17-05:00
9	ccb532ac-1308-fa17-5b1f-2c70334758e2	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1984-07-22T03:38:41-05:00	1984-07-22T04:09:03-05:00
10	ccb532ac-1308-fa17-49bf-5e3614d04200	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1985-07-28T03:38:41-05:00	1985-07-28T04:16:30-05:00
11	ccb532ac-1308-fa17-3921-6c5d0fa4b5da	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1989-02-05T03:38:41-05:00	1989-02-05T04:22:01-05:00
12	ccb532ac-1308-fa17-84d7-f2b707b20868	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	1997-07-20T03:38:41-05:00	1997-07-20T03:53:41-05:00
13	ccb532ac-1308-fa17-bbb5-a2b66150683d	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2004-05-16T03:38:41-05:00	2004-05-16T04:09:42-05:00
14	ccb532ac-1308-fa17-e434-1479d89ad072	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2009-02-20T09:09:08-05:00	2009-02-20T09:53:08-05:00
15	ccb532ac-1308-fa17-e8c7-0d98e3dc1036	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2009-04-15T08:29:41-04:00	2009-04-15T08:44:41-04:00
16	ccb532ac-1308-fa17-2cf8-98b6c65c08ad	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	EMER	2010-12-12T03:38:41-05:00	2010-12-12T04:38:41-05:00
17	ccb532ac-1308-fa17-d131-b4208115eafa	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	EMER	2010-12-19T03:38:41-05:00	2010-12-19T04:38:41-05:00
18	ccb532ac-1308-fa17-dc7e-8ec88b49dd0d	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2011-03-14T09:10:27-04:00	2011-03-14T14:48:17-04:00
19	ccb532ac-1308-fa17-2a09-5c31d0d03ee33	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2012-04-29T04:38:41-04:00	2012-04-29T05:45:28-04:00
20	ccb532ac-1308-fa17-3c61-d89e5a023eeb	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2013-02-10T03:38:41-05:00	2013-02-10T04:31:19-05:00
21	ccb532ac-1308-fa17-992e-d0110a945416	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2013-04-28T04:38:41-04:00	2013-04-28T05:32:16-04:00
22	ccb532ac-1308-fa17-defe-225b60560961	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	IMP	2013-08-25T04:38:41-04:00	2013-09-02T12:04:03-04:00
23	ccb532ac-1308-fa17-8d6d-a8eb2b1bcd9b	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2013-09-15T04:38:41-04:00	2013-09-15T05:37:10-04:00
24	ccb532ac-1308-fa17-5797-77137ba92f69	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2014-05-11T04:38:41-04:00	2014-05-11T05:23:40-04:00
25	ccb532ac-1308-fa17-0e0b-5dd75d6530e5	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2014-12-07T03:38:41-05:00	2014-12-07T04:09:52-05:00
26	ccb532ac-1308-fa17-d987-ef5aa987c9c9	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2015-01-18T03:38:41-05:00	2015-01-18T04:31:28-05:00
27	ccb532ac-1308-fa17-ca14-13680a0a7cf8	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2016-01-24T03:38:41-05:00	2016-01-24T04:22:24-05:00
28	ccb532ac-1308-fa17-859b-7ba14b4e9174	urn:uuid:ccb532ac-1308-fa17-ac8f-ed65b312345f	AMB	2016-08-08T03:38:41-05:00	2016-08-08T04:37:55-05:00

5. Discussion

5.1 Course Concepts in Practice

Pipeline Orchestration (Airflow): Airflow's DAG-based approach proved essential for managing complex task dependencies. The ShortCircuitOperator elegantly handles conditional execution (skip if no files) more cleanly than traditional if-else logic. The PostgreSQL metadata database provides complete lineage tracking - every task execution is logged with timestamps, duration, and status for audit purposes.

Data Pipeline Patterns: The staging/processing folder pattern is a classic solution to the producer-consumer problem in data engineering. Combined with atomic file moves (`shutil.move`), it ensures ACID-like properties in a file-based system. The archiving strategy implements "write-once, read-many" principle, crucial for audit trails.

Distributed Processing (PySpark): The project demonstrates how Spark's distributed computing model addresses healthcare data volume challenges. By partitioning data across executors, Spark enables horizontal scalability. However, for the test dataset (142 MB), Spark initialization overhead (~3-4 seconds) is significant. In production with terabytes of data, this overhead becomes negligible and parallel processing benefits dominate.

5.2 Design Trade-offs

Trade-off 1: Docker vs. Cloud-Native

- Decision: Docker Compose for local deployment
- Rationale: Reproducibility and zero cloud costs
- Trade-off: Limited scalability vs. Kubernetes or managed services (AWS EMR, Google Dataproc)
- Impact: Suitable for development/testing, requires migration for production scale

Trade-off 2: CSV vs. Parquet

- Decision: Output to CSV format
- Rationale: Universal compatibility with analytics tools
- Trade-off: Larger file size and slower queries vs. columnar formats (Parquet, ORC)

Trade-off 3: Batch vs. Streaming

- Decision: Batch processing every 5 minutes
- Rationale: Simpler implementation, sufficient for analytics use cases
- Trade-off: 5-minute latency vs. real-time streaming (Kafka + Spark Streaming)
- Impact: Acceptable for population health analytics, insufficient for real-time clinical alerts

5.3 Challenges and Surprises

Challenge 1: Race Conditions

Initial design had all tasks reading from the same `fhir/` folder. When new files arrived during processing, the `cleanfolder` task would miss them, leading to inconsistent batches. The staging/processing folder pattern solution is standard data engineering practice but wasn't obvious initially, demonstrating how production systems require thinking beyond happy-path scenarios.

Challenge 2: Cloud Environment Limitations

Running Airflow directly on the cloud free tier wasn't feasible due to strict resource and service limitations. Even when I tried running it on a VM (EC2 instance), the free-tier machines didn't have enough compute power to handle Airflow and Spark together. Because of these limits, I shifted to using the official Airflow Docker image locally, which allowed me to replicate a more complete environment.

Surprise:

PySpark Nested Field Complexity FHIR's deeply nested structure (5+ levels) required using Spark SQL expressions (`expr("name[0].given[0]")`) rather than DataFrame methods. The initial approach using `.getItem(0).getField("given")` failed on records where the name field was a string

instead of an array, demonstrating the importance of defensive programming with healthcare data.

5.4 Future Improvements

Data Quality Validation: Add validation tasks using Great Expectations to check for missing patient IDs, invalid dates, and referential integrity (e.g., conditions reference valid patients). Since healthcare analytics directly impacts patient care, automated quality checks should fail the pipeline when critical data issues are detected.

Stream Processing: Replace the current 5-minute batch schedule with real-time streaming using Kafka and Spark Structured Streaming. This would reduce latency from minutes to seconds, enabling real-time clinical dashboards and immediate alerting for critical patient conditions.

Monitoring Dashboard: Add a Grafana dashboard displaying pipeline execution times, record counts per run, and error rates over time. Integrate Slack notifications to alert the team when pipelines fail or when data quality issues are detected, replacing manual log checking.

5.5 Privacy and Access Control

Healthcare data requires stringent privacy controls per HIPAA regulations.

Current Implementation:

- Data stored locally in Docker volumes (not exposed to network)
- No PHI (Protected Health Information) in logs or error messages
- Synthetic data only (Synthea) - no real patient information

Production Requirements:

- Encryption: At-rest (disk encryption) and in-transit (TLS)
- Access Control: Role-based access control (RBAC) in Airflow
- Audit Logging: All data access logged with user, timestamp, action
- De-identification: Remove or hash direct identifiers (names, SSNs)
- Retention Policies: Automated deletion after 6-year retention period
- Compliance: Business Associate Agreement (BAA) with cloud providers

For production deployment, patient identifiers should be hashed using SHA-256 and names should be redacted or replaced with pseudonyms. The current implementation using synthetic data serves as a foundation that would require these additional controls for real patient data.

6. Conclusion

This project demonstrates that effective workflow orchestration is fundamental to building production-ready data pipelines. Apache Airflow's DAG-based orchestration enables complex multi-stage ETL workflows with proper task dependencies, conditional execution, and automatic retry mechanisms that would be extremely difficult to implement with simple scripts. The key takeaway is that orchestration frameworks transform fragile, error-prone scripts into reliable, monitored, and maintainable data pipelines.

The staging/processing folder pattern, ShortCircuitOperator for conditional execution, and timestamp-based archiving illustrate how orchestration tools enable sophisticated pipeline patterns beyond basic task sequencing. The pipeline implements a medallion architecture pattern, transforming raw FHIR bundles (bronze layer) into cleaned, structured CSV tables (silver layer) ready for analytics. These workflow orchestration principles - idempotency, atomic operations, complete audit trails, and graceful error handling - are transferable across domains, whether processing healthcare data, financial transactions, or IoT sensor streams. The combination of Airflow's orchestration with PySpark's distributed processing creates a scalable foundation that handles both the complexity of multi-step workflows and the volume challenges of big data.

Next Improvement: With more time, the immediate priority would be integrating data quality validation directly into the orchestrated workflow using Great Expectations. This would add a validation task between extraction and output, with Airflow sensors monitoring quality metrics and triggering alerts when thresholds are breached. This enhancement would demonstrate how orchestration frameworks enable ETL along with complete data quality monitoring and governance workflows essential for production healthcare systems.

The code for the repo can be found in the GitHub repo -

<https://github.com/vedanthirekar/healthcare-data-pipeline>

The repo has in-depth implementation steps and the rest of the details

7. References

1. Apache Airflow Documentation. (2024). "Airflow 2.10 Documentation." Apache Software Foundation. <https://airflow.apache.org/docs/>
2. Apache Spark Documentation. (2024). "Spark SQL, DataFrames and Datasets Guide." Apache Software Foundation. <https://spark.apache.org/docs/3.5.0/sql-programming-guide.html>
3. HL7 International. (2024). "FHIR R4 Specification." Health Level Seven International. <https://hl7.org/fhir/R4/>
4. Synthea. (2024). "Synthetic Patient Generator." MITRE Corporation. <https://synthetichealth.github.io/synthea/>
5. Docker Inc. (2025). "Docker Compose Documentation." <https://docs.docker.com/compose/>
6. Kleppmann, M. (2017). "Designing Data-Intensive Applications." O'Reilly Media.
7. Reis, J., & Housley, M. (2022). "Fundamentals of Data Engineering." O'Reilly Media.
8. U.S. Department of Health & Human Services. (2013). "HIPAA Privacy Rule." <https://www.hhs.gov/hipaa/>
9. Synthetic Data generation using Synthea - <https://github.com/synthetichealth/synthea>
10. What is Medallion Architecture - <https://www.databricks.com/glossary/medallion-architecture>